

Inspection / Review

This section describes the formal Inspection effort carried out on the software artifacts and source code of the application.

The inspection was conducted systematically following the Fagan Inspection methodology, operating as a proactive process to identify and locate defects in development documents across various phases of the software development lifecycle. By utilizing a multi-pass approach (individual preparation followed by group review) and standardized checklists, the methodology ensured rigorous evaluation prior to release.

All six members of the team took part in the inspection efforts. To maintain structural integrity during the review process, specific Fagan roles were assigned as follows:

Role	Team Member	Responsibilities
Moderator	Antonio Hus	Distributing materials, scheduling/leading the session, and ensuring post-meeting rework/corrections.
Secretary	Ariana Ilieș	Documenting critical observations and the final conclusions of the meeting.
Reader	Mara Ielciu	Guiding the team through the documentation during the meeting.
Inspector	Aris Horvat	Critical evaluation and defect identification.
Inspector	Costin Joldeș	Critical evaluation and defect identification.
Inspector	Patricia Manciu	Critical evaluation and defect identification.

The total time invested by the team across individual preparation and the formal meeting was approximately six hours. This exceeded the standard 90-120 minute Fagan window due to the considerable depth of the codebase and the comprehensive nature of the accompanying documentation.

Scope

The inspection comprehensively covered the project artifacts across three primary development phases to ensure alignment between specification and implementation:

Phase	Artifacts Inspected	Key Focus Areas
-------	---------------------	-----------------

Requirements	GymTrackerPro_Problem_Statement.pdf	Completeness, functional/non-functional clarity.
Design	GymTrackerPro_Problem_Design.pdf	Layered architecture, error handling strategy, design pattern application.
Implementation	GymTrackerPro Next.js Application	Decision logic, error branching, bounded loops, parameter usage.

Inspection Artifacts

The comprehensive documentation of this process was compiled into a single, formal 37-page Inspection Document. This primary artifact strictly follows the standardized inspection forms and checklists provided during the course seminary and laboratory sessions.

Methodology

The evaluation strictly adhered to the structured phases of the Fagan methodology:

- **Planning:** The moderator defined the team structure, assigned distinct roles, and distributed the source code and artifacts in advance.
- **Individual Preparation:** Each inspector independently reviewed the provided documentation against standard checklists to comprehend the system and identify potential defects.
- **Inspection Meeting:** Conducted online, the session progressed sequentially through the three phases (Requirements, Design, Code). Each inspector presented their individual observations, which were openly discussed and officially documented by the secretary.
- **Rework:** The respective authors addressed all recorded observations, implementing necessary corrections and logic updates.
- **Follow-up:** A verification pass was conducted to ensure all identified errors were successfully eliminated without introducing new structural defects.

Observations and Findings

The inspection yielded a highly positive overall assessment. The requirements were deemed unambiguous and well-prioritized, the design demonstrated a clear architectural and error-handling strategy, and the source code clearly mirrored the design specifications.

However, this rigorous approach successfully surfaced four specific observations that required attention:

- **Typographical Error:** A minor typographical defect was located in requirement EXM-01.
- **Documentation Ambiguity (Dates):** A notation inconsistency was observed between modules WSM-01 and RPG-01 regarding date formatting.
- **Inconsistent Interface Guarding:** The member layout was missing an explicit authentication redirect that was present in the admin layout.
- **State Synchronization Gap:** A logical flaw was identified regarding suspended users. The system did not explicitly re-check suspension status on subsequent requests for members holding an active session cookie, allowing a suspended member to continue accessing protected routes.

Rework and Resolution

Following the inspection meeting, the authors successfully completed the *Rework* and *Follow-up* phases to address all findings:

- **Clarification of Constraints:** The date format inconsistency was resolved by clarifying the specification rather than altering code: YYYY-MM-DD is utilized internally for validation/storage, whereas DD-MM-YYYY is strictly the user interface display format.
- **Requirement Corrections:** The typographical error in EXM-01 was corrected. Additionally, a new requirement, AUTH-07, was formally added to the specification to explicitly capture the necessary system behavior for active suspension checks.
- **Implementation Fixes:**
 - An explicit authentication guard was added to the member layout to match the admin layout.
 - A live database check of the member's active status was implemented on every page rendered to enforce immediate suspension.
 - Route protection middleware was updated to read the active status at login and proactively redirect suspended members to an unauthorized page prior to reaching member routes.

Exploratory Testing

This section describes the Exploratory Testing effort carried out on the GymTracker Pro application.

Unlike automated testing which relies on predefined test cases, this testing was heuristic-driven and focused on active discovery. To provide structure and measurable tracking to this effort, the testing was conducted using the Session-Based Test Management (SBTM) methodology.

Under this methodology, the testing was divided into dedicated, uninterrupted 60-minute exploratory sessions. Each session was guided by a specific "charter" that defined the target, the approach, and the exact risks or behaviors the tester was looking to uncover. The testing operated entirely from the perspective of the end-users (Gym Administrator and Gym Member personas), utilizing the live application environment (via Docker) to observe actual system behavior, identify edge cases, and discover inconsistencies that structural or automated testing might miss.

Scope

Testing covered the entire application lifecycle across six distinct operational areas:

Area	Features Tested
Authentication & Authorization	Gym Administrator provisioning, member login/logout, role-based access control, session cookie management, suspended account routing.
User Management	Member CRUD operations, form validation, search filtering, status toggling, detailed read-only views.

Exercise Management	Exercise CRUD operations, enum mappings (muscle groups, equipment), search trimming, archiving, constraint testing.
Workout Session Management	Cross-functional CRUD combining Users, Exercises, and Sessions. Complex form validation (sets, reps, weights, duration constraints).
Progress Report Generation	Gym Administrator and Gym Member report views, date range filtering, aggregate calculations (volume, duration), layout and data formatting.
Regression & Bug Verification	Systematic re-testing of all reported defects and edge cases from previous sessions to ensure complete resolution without side effects.

Test Artifacts

A total of 6 distinct exploratory sessions were conducted and documented using the SBTM reporting structure. Each session produced a log containing Setup details, Data used, Test observations, Bugs found, and Questions raised

Tester	Charter	Duration (Hours)
Antonio Hus	Explore the Authentication and Authorization mechanisms by acting as both admin and member personas to discover vulnerabilities in role-based access control, session management, and account provisioning.	1.09
Ariana Ilieș	Explore the User Management module by attempting to create, update, and delete user profiles using unexpected or edge-case data inputs to discover data validation flaws and state inconsistencies.	0.93
Costin Joldeș	Explore the Exercise Management features by attempting to create, update, and delete exercises using unexpected or edge-case data inputs to discover data validation flaws and state inconsistencies.	0.98
Mara Ielciu	Explore the Workout Session cross-functionality by attempting to create, update, and delete workout sessions using unexpected or edge-case data inputs to discover data validation flaws and state inconsistencies.	1.13
Aris Horvat	Explore the Progress Report Generation for both Admin and Member views by generating reports with varying data and date ranges to discover calculation inaccuracies and layout issues.	0.85
Patricia Manciu	Explore the recently deployed bug fixes and unresolved questions from previous sessions by re-testing the original	0.95

	failure scenarios to discover incomplete resolutions and unintended side-effects.	
--	---	--

Methodology

Session-Based Test Management (SBTM)

The exploratory testing effort was managed using the SBTM framework, applying the following core principles:

- **Charter-Driven Exploration:** Testers were not given step-by-step scripts. Instead, they were given a single-sentence charter and utilized their domain knowledge to creatively stress-test the application.
- **Persona Testing:** Testers actively switched between 'Admin' and 'Member' roles, often using parallel browser sessions or incognito windows, to ensure strict separation of concerns and data privacy.
- **Active Questioning:** When testers encountered ambiguous behavior (e.g., standard HTML validation vs. custom field errors, or missing read-only views), they logged them as "Questions" rather than immediate failures, prompting team-wide product decisions.
- **Dedicated Regression:** The final session was entirely dedicated to verifying the outputs of the previous five sessions, proving the feedback loop and accountability of the SBTM process.

Observations and Findings

The exploratory nature of this testing successfully uncovered 25 bugs, usability issues, and documentation flaws that escaped earlier testing phases.

Key themes in the findings included:

- **Silent Validation Failures:** Multiple complex forms (Exercise Description, Workout Session Notes, Workout Session Exercises) correctly identified when data was invalid (e.g., notes over 1024 characters, negative weight values) and blocked submission, but failed to render the specific field-level error to the user, resulting in a confusing UX.
- **Date Boundary and Formatting Issues:** The progress report generation logic failed to fetch data if the "Start Date" and "End Date" were the exact same day, ignoring valid same-day sessions. Furthermore, the report cards rendered dates in MM/DD/YYYY format, breaking consistency with the DD/MM/YYYY format used elsewhere in the system.
- **Documentation & Seed Data Mismatch:** The initial administrator password provided in the documentation (admin) did not match the strict schema requirements (8+ chars, uppercase, number, special char) enforced by the application, causing immediate login failures on fresh setups.
- **UI Inconsistencies:** Information categorization was occasionally misnamed (e.g., "Date of Birth" placed under a "Contact" section), and the application relied heavily on default HTML form warnings rather than utilizing the custom UI validation errors built for the system.

Resolution and Follow-Up

During the final Regression session, it was verified that all 25 reported bugs were fixed by the development team. Critical updates included adding noValidate tags to all forms to force custom field-level error rendering, fixing date inclusiveness for reports, correcting layout misnomers, and adding a much-needed demo-data seeding script to facilitate easier testing environments moving forward.

Frontend (Web) Testing

This document describes the Frontend Testing effort carried out on the User Interface (UI) layer of the GymTracker Pro application.

The primary objective was to verify that each component renders correctly and behaves exactly as intended when subjected to real user interactions.

The testing environment was completely isolated from the backend, as every Server Action was strictly mocked. The testing boundary operated entirely between the Component, the React DOM, and User Interaction (simulated via userEvent), utilizing Jest and React Testing Library (RTL) to assert on visible DOM outputs rather than internal implementation details.

Scope

Following a strict bottom-up testing order, the testing scope covered four distinct architectural layers of the frontend, ensuring each level was independently verified before being composed into the layer above:

Layer	Testing Focus	Example Components
Atoms	Single-responsibility UI elements. Tested for prop variants, disabled/loading states, and accessibility markers. No server actions.	Button, Input, Badge, ErrorMessage
Molecules	Composed UI elements with local state and validation. Tested for interaction flows and conditional rendering.	FormField, SearchBar, PaginationControls
Organisms	Feature-level components. Tested for full user workflows: idle states, happy paths, error handling, and recovery (with mocked server actions).	ExerciseForm, MemberList
Pages	Top-level routing components. Tested for layout rendering, routing triggers, and cross-organism data threading.	ExercisesPage, Dashboard

Test Artifacts

Test Files

A total of 21 test files (.test.tsx) were authored, one per module under test. The files collectively contain **365 automated tests**.

XLSX Documentation

A total of **57 structured XLSX documentation forms** were authored, covering the above-mentioned modules. Each form detailed the component's preconditions, props, expected output, and test cases.

Methodology

Decision-Point Coverage

Test cases were derived by enumerating every decision point (conditional renders, ternary operators, optional props) within a component.

Semantic DOM Querying

Assertions were strictly tied to what a real user or screen reader could observe (text content, roles, ARIA attributes). The use of internal component state or structural data-testid selectors was prohibited unless absolutely necessary.

Error Recovery Verification

For any component interacting with a server action, tests explicitly verified that upon a mocked network failure or rejection, the UI presented a clear error message and successfully re-enabled the form for subsequent user attempts.

Observations and Findings

The comprehensive execution of the 365 test cases resulted in a 100% coverage and pass rate against the initial implementation, with no structural, logical, or accessibility defects identified. Rather than surfacing bugs, the rigorous, bottom-up testing effort successfully validated the robustness and architectural integrity of the frontend layer.